Funções Simples e Algoritmos Numéricos

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Introdução às Funções
2. Alternativas em Funções Numéricas
3. Algoritmos Numéricos e Testes
4. Afetação Paralela

Da Aula 3 para a Aula 4

Na Aula 3 trabalhámos:

- o estado das variáveis durante a execução;
- a composição sequencial de instruções;
- caminhos alternativos com `if`, `elif` e `else`.

Hoje vamos usar essas ideias para:

- definir funções simples;
- distinguir parâmetros, argumentos e resultados;
- construir pequenos algoritmos numéricos;
- testar diferentes caminhos de uma função.

Pergunta-guia

Que estado do programa muda em cada passo?

Porquê Definir Funções?

```
1 base = 8
2 altura = 5
3 area = base * altura / 2
4 print(area)
```

- Este programa calcula uma área para valores concretos.

Porquê Definir Funções?

```
1 base = 8
2 altura = 5
3 area = base * altura / 2
4 print(area)
```

- Este programa calcula uma área para valores concretos.
- Uma função dá um **nome** ao cálculo.

Porquê Definir Funções?

```
1 base = 8
2 altura = 5
3 area = base * altura / 2
4 print(area)
```

- Este programa calcula uma área para valores concretos.
- Uma função dá um **nome** ao cálculo.
- Os valores podem variar entre chamadas.

Porquê Definir Funções?

```
1 base = 8
2 altura = 5
3 area = base * altura / 2
4 print(area)
```

- Este programa calcula uma área para valores concretos.
- Uma função dá um **nome** ao cálculo.
- Os valores podem variar entre chamadas.
- O resultado pode ser usado noutros cálculos, sem repetir o código.

Anatomia de uma Função

```
1 def area_triangulo(base: float, altura: float) -> float:
2     area = base * altura / 2
3     return area
```

- `def` inicia a definição; `area_triangulo` é o nome escolhido.

Anatomia de uma Função

```
1 def area_triangulo(base: float, altura: float) -> float:
2     area = base * altura / 2
3     return area
```

- `def` inicia a definição; `area_triangulo` é o nome escolhido.
- `base` e `altura` são **parâmetros**: nomes disponíveis no corpo.

Anatomia de uma Função

```
1 def area_triangulo(base: float, altura: float) -> float:
2     area = base * altura / 2
3     return area
```

- `def` inicia a definição; `area_triangulo` é o nome escolhido.
- `base` e `altura` são **parâmetros**: nomes disponíveis no corpo.
- O corpo indentado descreve o cálculo.

Anatomia de uma Função

```
1 def area_triangulo(base: float, altura: float) -> float:
2     area = base * altura / 2
3     return area
```

- `def` inicia a definição; `area_triangulo` é o nome escolhido.
- `base` e `altura` são **parâmetros**: nomes disponíveis no corpo.
- O corpo indentado descreve o cálculo.
- `return` termina a chamada e produz o resultado.

Anatomia de uma Função

```
1 def area_triangulo(base: float, altura: float) -> float:
2     area = base * altura / 2
3     return area
```

- `def` inicia a definição; `area_triangulo` é o nome escolhido.
- `base` e `altura` são **parâmetros**: nomes disponíveis no corpo.
- O corpo indentado descreve o cálculo.
- `return` termina a chamada e produz o resultado.
- As anotações indicam os tipos esperados; não substituem os testes.

Chamar uma Função

```
1 a1 = area_triangulo(8, 5)
2 a2 = area_triangulo(3.5, 2)
3 dobro = 2 * area_triangulo(4, 6)
4
5 print(a1)      # 20.0
6 print(a2)      # 3.5
7 print(dobro)   # 24.0
```

- `8` e `5` são os **argumentos** da primeira chamada.

Chamar uma Função

```
1 a1 = area_triangulo(8, 5)
2 a2 = area_triangulo(3.5, 2)
3 dobro = 2 * area_triangulo(4, 6)
4
5 print(a1)      # 20.0
6 print(a2)      # 3.5
7 print(dobro)   # 24.0
```

- `8` e `5` são os **argumentos** da primeira chamada.
- Durante essa chamada, `base == 8` e `altura == 5`.

Chamar uma Função

```
1 a1 = area_triangulo(8, 5)
2 a2 = area_triangulo(3.5, 2)
3 dobro = 2 * area_triangulo(4, 6)
4
5 print(a1)      # 20.0
6 print(a2)      # 3.5
7 print(dobro)   # 24.0
```

- `8` e `5` são os **argumentos** da primeira chamada.
- Durante essa chamada, `base == 8` e `altura == 5`.
- A chamada é uma expressão: pode surgir à direita de uma afetação ou dentro de outra expressão.

Chamar uma Função

```
1 a1 = area_triangulo(8, 5)
2 a2 = area_triangulo(3.5, 2)
3 dobro = 2 * area_triangulo(4, 6)
4
5 print(a1)      # 20.0
6 print(a2)      # 3.5
7 print(dobro)   # 24.0
```

- `8` e `5` são os **argumentos** da primeira chamada.
- Durante essa chamada, `base == 8` e `altura == 5`.
- A chamada é uma expressão: pode surgir à direita de uma afetação ou dentro de outra expressão.
- Cada chamada começa com novos valores para os parâmetros.

Função e Programa Cliente

Função: calcular

```
1 def area_triangulo(base, altura):
2     return base * altura / 2
```

Programa cliente: interagir

```
1 b = float(input("Base: "))
2 h = float(input("Altura: "))
3 resultado = area_triangulo(b, h)
4 print("Area:", resultado)
```

Função e Programa Cliente

Função: calcular

```
1 def area_triangulo(base, altura):
2     return base * altura / 2
```

Programa cliente: interagir

```
1 b = float(input("Base: "))
2 h = float(input("Altura: "))
3 resultado = area_triangulo(b, h)
4 print("Area:", resultado)
```

- A função não precisa de saber de onde vieram os dados nem como o resultado será apresentado.

Função e Programa Cliente

Função: calcular

```
1 def area_triangulo(base, altura):
2     return base * altura / 2
```

Programa cliente: interagir

```
1 b = float(input("Base: "))
2 h = float(input("Altura: "))
3 resultado = area_triangulo(b, h)
4 print("Area:", resultado)
```

- A função não precisa de saber de onde vieram os dados nem como o resultado será apresentado.
- Separar cálculo e interação torna o código mais reutilizável e mais fácil de testar.

Exercício: Definir uma Função Numérica

Problema

Defina `distancia(x1, x2)`, que recebe duas posições numa reta e devolve a distância entre elas.

Perguntas

- Que expressão calcula a distância sem depender da ordem das posições?
- Quais devem ser os parâmetros e o resultado?
- Que chamadas permitem testar posições iguais e posições por ordem inversa?

Uma Solução Possível: Distância numa Reta

Problema: devolver a distância entre as posições `x1` e `x2`.

```
1 def distancia(x1: float, x2: float) -> float:
2     return abs(x2 - x1)
3
4 assert distancia(2, 7) == 5
5 assert distancia(7, 2) == 5
6 assert distancia(3.5, 3.5) == 0
```

- `abs` elimina a dependência da ordem dos argumentos.
- Os testes cobrem ordem crescente, ordem decrescente e posições iguais.

Contrato Informal de uma Função

Antes de programar, responda a quatro perguntas:

1. **Nome:** que cálculo representa a função?
2. **Entradas:** que parâmetros recebe e que valores são válidos?
3. **Resultado:** que valor devolve?
4. **Casos de teste:** que exemplos normais e fronteiras confirmam o comportamento?

Contrato Informal de uma Função

Antes de programar, responda a quatro perguntas:

1. **Nome:** que cálculo representa a função?
2. **Entradas:** que parâmetros recebe e que valores são válidos?
3. **Resultado:** que valor devolve?
4. **Casos de teste:** que exemplos normais e fronteiras confirmam o comportamento?

Exemplo

`distancia(x1, x2)` recebe duas posições numéricas e devolve um número não negativo.

Outline

1. Introdução às Funções
2. Alternativas em Funções Numéricas
3. Algoritmos Numéricos e Testes
4. Afetação Paralela

Resolvedor de Equações de 2.º Grau

Problema: Dada uma equação quadrática na forma normal

$$ax^2 + bx + c = 0$$

encontrar e imprimir os valores de x que a satisfazem. Usar a fórmula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

NB.: Esta fórmula foi deduzida pela primeira vez pelo matemático árabe Al-Khwārizmī (780–850), de cujo nome deriva o termo "algoritmo".

Resolvedor de Equações de 2.º Grau — Análise

```python
import math

def solve_quadratic():
    print("Welcome to Al-Khwarizi 1.0")
    print("Solving ax^2+bx+c==0")
    a = float(input("intro a = "))
    b = float(input("intro b = "))
    c = float(input("intro c = "))
    d = b*b - 4*a*c
    if d < 0:
        print("No solution!")
    elif d == 0:
        x = -b / (2*a)
        print("One root x =", x)
    else:
        x1 = (-b - math.sqrt(d)) / (2*a)
        x2 = (-b + math.sqrt(d)) / (2*a)
        print("First root x1 =", x1)
        print("Second root x2 =", x2)
```

Programar: Resolvedor de Equações de 2º Grau

Construção Python `if` / `elif` / `else` (alternativas encadeadas)

```
1 if C1:
2     P1
3 elif C2:
4     P2
5 elif C3:
6     P3
7 else:
8     Pk
```

- Avalia as condições em ordem; executa o primeiro bloco cuja condição é verdadeira.
- Se nenhuma condição for verdadeira, executa o bloco `else` (opcional).

Outline

1. Introdução às Funções
2. Alternativas em Funções Numéricas
3. Algoritmos Numéricos e Testes
4. Afetação Paralela

Discriminante: Separar um Subproblema

```python
def discriminante(a: float,
                  b: float,
                  c: float) -> float:
    return b*b - 4*a*c

def numero_raizes_reais(a: float,
                        b: float,
                        c: float) -> int:
    if a == 0:
        raise ValueError("a nao pode ser 0")
    d = discriminante(a, b, c)
    if d < 0:
        return 0
    elif d == 0:
        return 1
    else:
        return 2
```

Discriminante: Separar um Subproblema

```python
def discriminante(a: float,
                  b: float,
                  c: float) -> float:
    return b*b - 4*a*c

def numero_raizes_reais(a: float,
                        b: float,
                        c: float) -> int:
    if a == 0:
        raise ValueError("a nao pode ser 0")
    d = discriminante(a, b, c)
    if d < 0:
        return 0
    elif d == 0:
        return 1
    else:
        return 2
```

```python
assert discriminante(1, -3, 2) == 1

assert numero_raizes_reais(
    1, 0, 1) == 0
assert numero_raizes_reais(
    1, -2, 1) == 1
assert numero_raizes_reais(
    1, -3, 2) == 2
```

- Uma função pequena permite testar separadamente cada parte do algoritmo.
- Os três testes percorrem os três resultados possíveis.

Funções que Respondem a Perguntas

```
1 def e_positivo(x: float) -> bool:
2     return x > 0
3
4 def esta_no_intervalo(x: float,
5                         minimo: float,
6                         maximo: float)
                              -> bool:
7     return minimo <= x <= maximo
```

- Uma função pode devolver qualquer tipo de valor, incluindo `bool`.

```
1 if esta_no_intervalo(nota, 0, 20):
2     print("nota valida")
```

Funções que Respondem a Perguntas

```
1 def e_positivo(x: float) -> bool:
2     return x > 0
3
4 def esta_no_intervalo(x: float,
5                         minimo: float,
6                         maximo: float)
                              -> bool:
7     return minimo <= x <= maximo
```

- Uma função pode devolver qualquer tipo de valor, incluindo `bool`.
- O resultado de uma comparação já é `True` ou `False`.

```
1 if esta_no_intervalo(nota, 0, 20):
2     print("nota valida")
```

Funções que Respondem a Perguntas

```
1 def e_positivo(x: float) -> bool:
2     return x > 0
3
4 def esta_no_intervalo(x: float,
5                        minimo: float,
6                        maximo: float)
                             -> bool:
7     return minimo <= x <= maximo
```

- Uma função pode devolver qualquer tipo de valor, incluindo `bool`.
- O resultado de uma comparação já é `True` ou `False`.
- Estas funções podem ser chamadas diretamente numa condição.

```
1 if esta_no_intervalo(nota, 0, 20):
2     print("nota valida")
```

Exercício: Valor Absoluto sem `abs`

Problema

Defina `valor_absoluto(x)`, que devolve a distância de `x` a zero sem chamar **`abs`**.

Perguntas

- Que dois casos é necessário distinguir?
- O que deve acontecer para `x == 0`?
- Que três testes cobrem valores negativo, nulo e positivo?

Uma Solução Possível: Valor Absoluto

Problema: devolver a distância de `x` a zero sem usar **`abs`**.

```
1 def valor_absoluto(x: float) -> float:
2     if x < 0:
3         return -x
4     return x
5
6 assert valor_absoluto(-3) == 3
7 assert valor_absoluto(0) == 0
8 assert valor_absoluto(2.5) == 2.5
```

- O primeiro **`return`** trata o caso negativo e termina a chamada.
- Se a condição for falsa, a execução continua no segundo **`return`**.

Quociente e Resto

Para inteiros `a` e `b > 0`, a divisão inteira separa `a` em quociente e resto:

$$a = (a \,//\, b) \times b + (a \bmod b)$$

```python
def mostrar_divisao(a: int, b: int) -> None:
    quociente = a // b
    resto = a % b
    print("quociente:", quociente)
    print("resto:", resto)

mostrar_divisao(17, 5)  # quociente 3, resto 2
```

Quociente e Resto

Para inteiros `a` e `b > 0`, a divisão inteira separa `a` em quociente e resto:

$$a = (a \,//\, b) \times b + (a \bmod b)$$

```
1 def mostrar_divisao(a: int, b: int) -> None:
2     quociente = a // b
3     resto = a % b
4     print("quociente:", quociente)
5     print("resto:", resto)
6
7 mostrar_divisao(17, 5)  # quociente 3, resto 2
```

- Resto zero significa que a divisão é exata.

Exercício: Testar Divisibilidade

Problema

Defina `e_divisor(d, n)`, que devolve `True` quando `d` divide `n` exatamente.

Condições

- `d == 0` não representa uma divisão válida e deve produzir `False`;
- nos restantes casos, use o resto da divisão;
- teste um divisor, um não divisor e o caso `d == 0`.

Uma Solução Possível: Testar Divisibilidade

Problema: decidir se `d` é um divisor válido e exato de `n`.

```
1 def e_divisor(d: int, n: int) -> bool:
2     if d == 0:
3         return False
4     return n % d == 0
5
6 assert e_divisor(3, 12)
7 assert not e_divisor(5, 12)
8 assert not e_divisor(0, 12)
```

Compor Funções Simples

```
1 def tem_raiz_dupla(a: float,
2                    b: float,
3                    c: float) -> bool:
4     return discriminante(a, b, c) == 0
5
6 assert tem_raiz_dupla(1, -2, 1)
7 assert not tem_raiz_dupla(1, -3, 2)
```

- `tem_raiz_dupla` reutiliza o significado já dado a `discriminante`.

Compor Funções Simples

```
1 def tem_raiz_dupla(a: float,
2                    b: float,
3                    c: float) -> bool:
4     return discriminante(a, b, c) == 0
5
6 assert tem_raiz_dupla(1, -2, 1)
7 assert not tem_raiz_dupla(1, -3, 2)
```

- `tem_raiz_dupla` reutiliza o significado já dado a `discriminante`.
- O programa maior é construído combinando funções pequenas.

Compor Funções Simples

```
1 def tem_raiz_dupla(a: float,
2                    b: float,
3                    c: float) -> bool:
4     return discriminante(a, b, c) == 0
5
6 assert tem_raiz_dupla(1, -2, 1)
7 assert not tem_raiz_dupla(1, -3, 2)
```

- `tem_raiz_dupla` reutiliza o significado já dado a `discriminante`.
- O programa maior é construído combinando funções pequenas.
- Alterações ao cálculo do discriminante ficam concentradas numa só função.

Máximo de Três Valores — Problema

Problema

Definir `maximo3(a, b, c)`, que devolve o maior de três valores numéricos sem usar a função **`max`**.

Perguntas

- Que valor pode servir de melhor candidato inicial?
- Em que situações deve esse candidato ser substituído?
- Que casos testam empates e diferentes posições do máximo?

Uma Solução Possível: Máximo de Três Valores

Problema: devolver o maior de `a`, `b` e `c` sem usar **`max`**.

```
1  def maximo3(a: float,
2              b: float,
3              c: float) -> float:
4      maior = a
5      if b > maior:
6          maior = b
7      if c > maior:
8          maior = c
9      return maior
10
11 assert maximo3(7, 2, 5) == 7
12 assert maximo3(2, 7, 5) == 7
13 assert maximo3(2, 5, 7) == 7
14 assert maximo3(7, 7, 2) == 7
```

Outline

1. Introdução às Funções
2. Alternativas em Funções Numéricas
3. Algoritmos Numéricos e Testes
4. Afetação Paralela

Afetação Paralela

- A afetação paralela permite atribuir múltiplos valores em simultâneo:

```
1 x, y, z = 1.0, -1.0, 2.0
2 a, b = b, a  # troca sem var. temp.
3 quociente, resto = 17 // 5, 17 % 5
```

Afetação Paralela

- A afetação paralela permite atribuir múltiplos valores em simultâneo:

```
1 x, y, z = 1.0, -1.0, 2.0
2 a, b = b, a  # troca sem var. temp.
3 quociente, resto = 17 // 5, 17 % 5
```

- Como é executada:
  1. Calcula o valor de **cada** expressão à direita do =.
  2. Altera o conteúdo das variáveis à esquerda com o valor previamente calculado.

Afetação Paralela

- A afetação paralela permite atribuir múltiplos valores em simultâneo:

```
1 x, y, z = 1.0, -1.0, 2.0
2 a, b = b, a  # troca sem var. temp.
3 quociente, resto = 17 // 5, 17 % 5
```

- Como é executada:
  1. Calcula o valor de **cada** expressão à direita do =.
  2. Altera o conteúdo das variáveis à esquerda com o valor previamente calculado.
- "inicial" quer dizer "antes da afetação paralela ter lugar".

Afetação Paralela

- A afetação paralela permite atribuir múltiplos valores em simultâneo:

```
1 x, y, z = 1.0, -1.0, 2.0
2 a, b = b, a  # troca sem var. temp.
3 quociente, resto = 17 // 5, 17 % 5
```

- Como é executada:
  1. Calcula o valor de **cada** expressão à direita do `=`.
  2. Altera o conteúdo das variáveis à esquerda com o valor previamente calculado.
- "inicial" quer dizer "antes da afetação paralela ter lugar".
- `a, b = b, a` troca o valor de `a` com o valor de `b` sem usar variável temporária.

Erros Comuns e Depuração

Tipos de erro:

```
# SyntaxError: código mal escrito
if x = 5:     # erro: = em vez de ==
    pass

# NameError: variavel nao definida
print(y)      # NameError: y undefined

# TypeError: tipo errado
3 + [5]       # TypeError: int + list

# ZeroDivisionError
10 / 0        # ZeroDivisionError

# IndentationError: indentacao errada
def f():
x = 1         # IndentationError
```

Como depurar:

- Ler a mensagem de erro: diz o tipo e a linha
- **Print debugging**: adicionar `print()` para ver valores intermédios
- Testar com casos simples primeiro
- Verificar tipos com `type()`

```
# Exemplo de print debugging:
def dividir(a, b):
    print("argumentos:", a, b)
    if b == 0:
        print("divisor invalido")
        return None
    resultado = a / b
    print("resultado:", resultado)
    return resultado
```

Exercícios das Aulas 3 e 4

1. **Conversor de temperatura**: ler uma temperatura em Celsius e imprimir em Fahrenheit ($F = \frac{9}{5}C + 32$) e Kelvin ($K = C + 273.15$).
2. **Calculadora**: ler dois números e um operador (+, −, *, /); usar `if`/`elif` para calcular o resultado (tratando divisão por zero).
3. **Intervalo**: ler três inteiros lo, hi e x; verificar se $lo \leq x \leq hi$.
4. **Valor intermédio**: definir uma função que devolve o valor que não é mínimo nem máximo entre três números distintos.
5. **Equação quadrática**: definir uma função que devolve o número de raízes reais a partir de a, b e c.

Resumo: Construções Python das Aulas 3 e 4

Controlo de fluxo:

- `if`/`elif`/`else`: alternativa
- condições são avaliadas por ordem
- apenas o primeiro ramo verdadeiro é executado
- todos os caminhos devem produzir um estado válido

Dados:

- Tipos: `int`, `float`, `bool`
- Operadores: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Comparação: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Lógico: `and`, `or`, `not`

Funções:

- `def f(params): return val`
- parâmetros, argumentos e resultados
- chamadas como expressões
- Afetação paralela: `a, b = b, a`

Boas práticas:

- Escolher a construção de controlo pela forma do problema.
- Manter cada função pequena, testável e com uma responsabilidade clara.

Resumo da Aula 4

- Funções simples:
  - definição com `def`, parâmetros e argumentos;
  - resultado produzido por `return`;
  - chamadas usadas dentro de expressões.
- Algoritmos numéricos com alternativas e funções auxiliares.
- Testes com `assert` para casos normais, fronteiras e caminhos distintos.

Conclusão

Hoje vimos

- funções simples com parâmetros e resultado
- algoritmos numéricos com alternativas e funções auxiliares
- testes de caminhos e valores de fronteira com `assert`

A seguir

- ciclos `while` para repetir cálculos
- estado, condição de continuação e terminação
- algoritmos iterativos numéricos e respetivos invariantes